Explicit Architectural Weight Sharing in Bidirectional LSTMs

Overview

Standard BiLSTMs treat separate forward and backward parameter sets as definitional. We challenge this with a single insight: *the LSTM cell is direction-blind*. Direction, role, and language identity all live in the pipeline around the cell, not inside its weights. We validate this claim across two experiments with progressively harder sharing demands.

Base cell equation (shared across all experiments):

$$(h_t,\, c_t) \;=\; \varphi(x_t,\, h_{t-1},\, c_{t-1};\; \theta)$$

θ contains no notion of direction, role, or language. The following experiments demonstrate this concretely.

Experiment 1 Bidirectional Sentiment (SST-2)

1a. Architectural Change

In the standard BiLSTM, two calls to `_make_lstm_layer` produce two independent objects at two memory addresses:

```
fwd = _make_lstm_layer(d, h)
bwd = _make_lstm_layer(d, h)
# fwd IS NOT bwd
```

Our change adds a single `shared_layer` argument:

```
fwd = _make_lstm_layer(d, h)
bwd = _make_lstm_layer(d, h,
        shared_layer=fwd)
# fwd IS bwd. Same memory address.
```

This is the **complete architectural change**—one `if` clause at construction time. Not a regularization term; not a soft penalty. The same `nn.ParameterDict` object is referenced by both the forward and backward loops. Gradients from both directions accumulate into the same tensor automatically.

1b. Math

Standard BiLSTM (baseline)

Two independent parameter sets process the same sequence in opposite directions:

$$(h_t^f, c_t^f) = \varphi(x_t,\, h_{t-1}^f,\, c_{t-1}^f;\; \theta_f)$$

$$(h_t^b, c_t^b) = \varphi(x_t,\, h_{t+1}^b,\, c_{t+1}^b;\; \theta_b)$$

with $\theta_f \cap \theta_b = \emptyset$.

SharedWeight BiLSTM (ours)

One θ is reused; direction is determined entirely by loop indexing:

$$(h_t^f, c_t^f) = \varphi(x_t,\, h_{t-1}^f,\, c_{t-1}^f;\; \theta), \quad t = 1 \to T$$

$$(h_t^b, c_t^b) = \varphi(x_t,\, h_{t+1}^b,\, c_{t+1}^b;\; \theta), \quad t = T \to 1$$

Fusion: At each intermediate layer the full per-timestep outputs are added. The final sentence representation fed to the classifier uses the terminal hidden states from each direction:

$$\tilde{h} \;=\; h_T^f + h_0^b \;\in\; \mathbb{R}^h$$

Both directions live in $\mathrm{Im}(\varphi_\theta)$—the same latent space—so addition is geometrically meaningful. Separate θ_f, θ_b offer no such guarantee.

Gradient accumulation into shared θ:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \underbrace{\sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t^f} \frac{\partial h_t^f}{\partial \theta}}_{\text{forward stream}} + \underbrace{\sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t^b} \frac{\partial h_t^b}{\partial \theta}}_{\text{backward stream}}$$

Both streams regularize θ bidirectionally from epoch 1.

Parameter reduction: The Standard model uses PyTorch `nn.LSTM`, which maintains two bias vectors per gate (`bias_ih` and `bias_hh`):

$$|\theta_{\text{standard}}| = 2 \cdot 4h(d + h + 2)$$

The Shared model uses a custom `ParameterDict` cell with one bias per gate (matching Adigun's original implementation):

$$|\theta_{\text{shared}}| = 4h(d + h + 1)$$

The verified parameter counts (235,778 vs. 117,506) confirm 50.2% reduction. Note the two formulas differ by one bias term per gate — the reduction arises from halving the cell count, not from this implementation difference.

1c. Results

Setup: SST-2, 67,349 train / 872 val. GloVe 6B 100d frozen. Hidden dim 128. Adam, lr = 1e-3, patience = 3.

Metric	Standard	SharedBi
Best Val Accuracy	84.17%	83.03%
Trainable Params	235,778	117,506
Param Reduction	—	**50.2%**
Epochs to Converge	10	6
Epoch 1 Val Acc	77.29%	**80.16%**
Accuracy Retained	—	98.6%

Table 1: SST-2 sentiment classification results.

1d. Verdict

The LSTM cell is direction-blind. SharedBi recovers 98.6% of Standard BiLSTM accuracy at exactly 50% of the parameters. It starts *stronger* at epoch 1 (80.16% vs. 77.29%), confirming that bidirectional gradient pressure acts as regularization from the first update.

Experiment 2 Bidirectional Translation (Multi30k)

2a. Architectural Change

Standard NMT for two translation directions requires four separate LSTM cells—one encoder and one decoder per direction:

Role	Direction	Cell
Encoder	EN→DE	$\theta_{\text{enc}}^{\text{en2de}}$
Decoder	EN→DE	$\theta_{\text{dec}}^{\text{en2de}}$
Encoder	DE→EN	$\theta_{\text{enc}}^{\text{de2en}}$
Decoder	DE→EN	$\theta_{\text{dec}}^{\text{de2en}}$

Our change replaces all four with a single `LSTMCell` object:

```
self.cell = LSTMCell(emb_dim, hid_dim)
# enc_en2de IS dec_en2de IS
# enc_de2en IS dec_de2en
# All four roles. One object.
```

Role and language direction are determined by which embedding matrix (E_{EN} or E_{DE}) is fed in and which output head (FC_{DE} or FC_{EN}) reads out. The cell itself is untouched.

2b. Math

EN→DE path

Encoding English:

$$(h_t, c_t) = \varphi(E_{\text{EN}}(x_t^{\text{en}}),\ h_{t-1},\ c_{t-1};\ \theta)\,, \quad t = 1 \to T_{\text{src}}$$

Final state $h_{T_{\text{src}}}$ is the sentence representation.

Decoding into German:

$$(h_t, c_t) = \varphi(E_{\text{DE}}(\hat{y}_{t-1}),\ h_{t-1},\ c_{t-1};\ \theta)\,, \quad \hat{y}_t = \text{FC}_{\text{DE}}(h_t)$$

DE→EN path

Identical structure with embeddings and output heads swapped. The same θ encodes German and decodes English.

Joint loss and gradient accumulation

Both translation directions contribute to one weight update:

$$\mathcal{L} = \mathcal{L}_{\text{EN}\to\text{DE}} + \mathcal{L}_{\text{DE}\to\text{EN}}$$

$$\frac{\partial \mathcal{L}}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}_{\text{EN}\to\text{DE}}}{\partial \theta}}_{\text{EN enc + DE dec}} + \underbrace{\frac{\partial \mathcal{L}_{\text{DE}\to\text{EN}}}{\partial \theta}}_{\text{DE enc + EN dec}}$$

All four roles send gradient signals into the same θ each iteration.

Parameter reduction

$$|\theta_{\text{standard}}^{\text{LSTM}}| = 4 \cdot 4h(d + h + 1) \quad |\theta_{\text{shared}}^{\text{LSTM}}| = 4h(d + h + 1)$$

Exact 75% LSTM cell reduction. E_{EN}, E_{DE}, FC_{EN}, FC_{DE} remain language-specific and are not part of the sharing claim.

2c. Results

Setup: Multi30k, 29,000 train / 1,014 val / 1,000 test. Separate EN (7,704) and DE (9,597) vocabularies. Embed dim 256, hidden dim 512. Adam, lr = 1e-3, patience = 5. Training alternates EN→DE and DE→EN batches each iteration.

Metric	Exp1 (4 cells)	Exp2 (1 cell)
EN→DE BLEU	23.90	22.17
DE→EN BLEU	20.55	18.81
Avg BLEU	22.23	20.49
BLEU Retained	—	92.2%
Total Params	24,033,173	14,879,381
Total Reduction	—	38.1%
LSTM Cell Params	6,299,648	1,574,912
LSTM Cell Reduction	—	**75.0%**
LSTM Cell Count	4	**1**

Table 2: Multi30k EN↔DE translation results.

2d. Verdict

The LSTM cell is role-blind and language-blind. One shared θ simultaneously encodes English, decodes German, encodes German, and decodes English. This achieves a 75% reduction in LSTM cell parameters while retaining 92.2% average BLEU. The ∼1.7 BLEU cost quantifies the optimization burden of satisfying four directional and role constraints on a single parameter set. Total model reduction is 38.1% (not 75%) because embedding matrices and output heads remain language-specific by design—the claim is *precisely at the LSTM cell level.*

Unified Contribution

Property	Exp	Reduction	Cost
Direction-blind	SST-2	50% params	1.1% acc
Role-blind	NMT	75% LSTM cells	1.74 BLEU
Language-blind	NMT	75% LSTM cells	1.74 BLEU

Table 3: Three confirmed properties of a shared θ. Role-blind and language-blind are both verified by Exp2 vs Exp1; the 1.74 BLEU cost (avg) is the joint price of satisfying both.

The Adigun–Kosko BBP framework established that error signals can flow bidirectionally through shared weights during learning. These experiments show the complementary result at inference time: a single θ can produce multiple directional trajectories simultaneously because direction, role, and language are properties of the pipeline, never the cell.

Limitations & Next Steps

The shared cell imposes a harder optimization burden; the ∼1.7 BLEU cost quantifies this. Sequential Python loops are ∼2× slower per epoch than cuDNN kernels which is a software limitation, not architectural.